Assignment 1

Satisfiability & SAT Solvers

Contents of this Report

- **Exercise 1:** The DPLL Algorithm — CNF, Inference Rules, Pseudocode & Worked Example
- **Exercise 2:** Z3 SMT Solver — Installation, Formula Analysis & Python Implementation

Contents

1 Propositional Logic & Conjunctive Normal Form **2**
- 1.1 Propositional Formulas . . . 2
- 1.2 Conjunctive Normal Form (CNF) . . . 2
- 1.3 Converting Any Formula to CNF . . . 2

2 The DPLL Algorithm **2**
- 2.1 Core Inference Rules . . . 2
  - 2.1.1 Unit Propagation . . . 3
  - 2.1.2 Pure Literal Elimination . . . 3
  - 2.1.3 Backtracking (Case Split) . . . 3
- 2.2 DPLL Pseudocode . . . 3
- 2.3 Worked Example . . . 4
- 2.4 Illustrations . . . 4
- 2.5 Completeness & Extension to CDCL . . . 4

3 Exercise 2 — Z3 SMT Solver **4**
- 3.1 What is Z3? . . . 5
- 3.2 Installation . . . 5
- 3.3 The Formula: $p \wedge \neg\neg(\neg q \wedge \neg\neg p)$ . . . 5
- 3.4 Python Implementation . . . 5
- 3.5 Expected Output . . . 6
- 3.6 Manual Verification . . . 6
- 3.7 Algorithm for DPLL-based Z3 Resolution . . . 6
- 3.8 Comparison: Linear Solver vs. DPLL vs. Z3 . . . 6

4 Terminal-Outputs & Submission **7**

5 Conclusion **8**

1. Propositional Logic & Conjunctive Normal Form

Before understanding the DPLL algorithm, it is essential to grasp *Conjunctive Normal Form* (CNF), the canonical representation on which DPLL operates.

1.1 Propositional Formulas

A propositional formula is built from Boolean variables (e.g. p, q, r) and the logical connectives $\neg$, $\wedge$, $\vee$, $\rightarrow$, $\leftrightarrow$. A **satisfying assignment** (or *model*) is a mapping of variables to $\{\mathit{True}, \mathit{False}\}$ that makes the formula evaluate to *True*.

1.2 Conjunctive Normal Form (CNF)

A formula is in **CNF** if it is a *conjunction* (**AND**) of *clauses*, where every clause is a *disjunction* (**OR**) of *literals*. A **literal** is a variable x or its negation $\neg x$.

Formally:

$$\underbrace{(l_{1,1} \vee \cdots \vee l_{1,k_1})}_{\text{clause 1}} \wedge \underbrace{(l_{2,1} \vee \cdots \vee l_{2,k_2})}_{\text{clause 2}} \wedge \cdots \wedge \underbrace{(l_{n,1} \vee \cdots \vee l_{n,k_n})}_{\text{clause } n}$$

Example CNF formula:

$$(A \vee B) \;\wedge\; (\neg A \vee C) \;\wedge\; (\neg B \vee \neg C)$$

1.3 Converting Any Formula to CNF

Any propositional formula can be mechanically converted to CNF:

- **Step 1 — Eliminate implications:** $A \rightarrow B \;\equiv\; \neg A \vee B$
- **Step 2 — Push negations inward** (De Morgan's Laws): $\neg(A \wedge B) \equiv \neg A \vee \neg B$, $\neg(A \vee B) \equiv \neg A \wedge \neg B$
- **Step 3 — Eliminate double negations:** $\neg\neg A \equiv A$
- **Step 4 — Distribute $\vee$ over $\wedge$:** $A \vee (B \wedge C) \equiv (A \vee B) \wedge (A \vee C)$

After these steps the formula is in CNF and ready for DPLL.

2. The DPLL Algorithm

The **Davis–Putnam–Logemann–Loveland (DPLL)** algorithm (1962) is a complete, backtracking-based procedure for deciding the satisfiability of CNF formulas. It underpins virtually all modern SAT solvers.

2.1 Core Inference Rules

DPLL applies three simplification strategies before any branching.

2.1.1 Unit Propagation

Rule: If a clause contains exactly one literal ℓ (a *unit clause*), then ℓ must be assigned *True*. All clauses containing ℓ are removed; $\neg\ell$ is removed from every remaining clause. Repeat until no unit clause exists.

Example:

$$(p) \;\wedge\; (\neg p \vee q) \;\wedge\; (\neg q \vee r) \xrightarrow{p=\top} (q) \;\wedge\; (\neg q \vee r) \xrightarrow{q=\top} (r) \xrightarrow{r=\top} \emptyset \;\Rightarrow\; \textbf{SAT}$$

2.1.2 Pure Literal Elimination

Rule: A literal ℓ is *pure* if $\neg\ell$ never appears in the formula. Assigning $\ell = \mathit{True}$ satisfies every clause containing it without breaking any other clause.

Example: $(A \vee B) \;\wedge\; (\neg B \vee C) \;\wedge\; (A \vee \neg C)$ — A is pure $\Rightarrow$ assign $A = \top$, remove clauses containing A; result: $(\neg B \vee C)$.

2.1.3 Backtracking (Case Split)

Rule: When neither UP nor PLE applies, choose an unassigned variable x and recurse on two branches: $x = \top$ and $x = \bot$. If both branches fail, the formula is **UNSAT**.

2.2 DPLL Pseudocode

Algorithm 1 DPLL(F, α) — Decide satisfiability of CNF formula F

Input: CNF formula F, partial assignment α
Output: **SAT** with model, or **UNSAT**

```
                                                    ▷ — Unit Propagation —
while ∃ unit clause (ℓ) in F do
    F ← simplify(F, ℓ)
    α ← α ∪ {var(ℓ) ↦ pol(ℓ)}
    if □ ∈ F then return UNSAT
    end if
end while
                                                    ▷ — Pure Literal Elimination —
for each pure literal ℓ in F do
    F ← simplify(F, ℓ)
    α ← α ∪ {var(ℓ) ↦ pol(ℓ)}
end for
                                                    ▷ — Base Cases —
if F = ∅ then return SAT, α
end if
if □ ∈ F then return UNSAT
end if
                                                    ▷ — Branching —
x ← chooseVariable(F)
if DPLL(simplify(F, x), α ∪ {x ↦ ⊤}) = SAT then return SAT
end if
if DPLL(simplify(F, ¬x), α ∪ {x ↦ ⊥}) = SAT then return SAT
end ifreturn UNSAT
```

2.3 Worked Example

We run DPLL on $\varphi = (A \vee B) \ \wedge \ (\neg A \vee C) \ \wedge \ (\neg B \vee \neg C)$.

Initial state. No unit clauses; no pure literals. Choose variable A.

Branch $A = \top$:

$$\{A, B\},\ \{\neg A, C\},\ \{\neg B, \neg C\} \xrightarrow{A=\top} \{C\},\ \{\neg B, \neg C\} \xrightarrow{\text{UP: } C=\top} \{\neg B\} \xrightarrow{\text{UP: } B=\bot} \emptyset$$

SAT Model: $A = \top,\ C = \top,\ B = \bot$

The second branch is never reached because the first already yields a model.

2.4 Illustrations

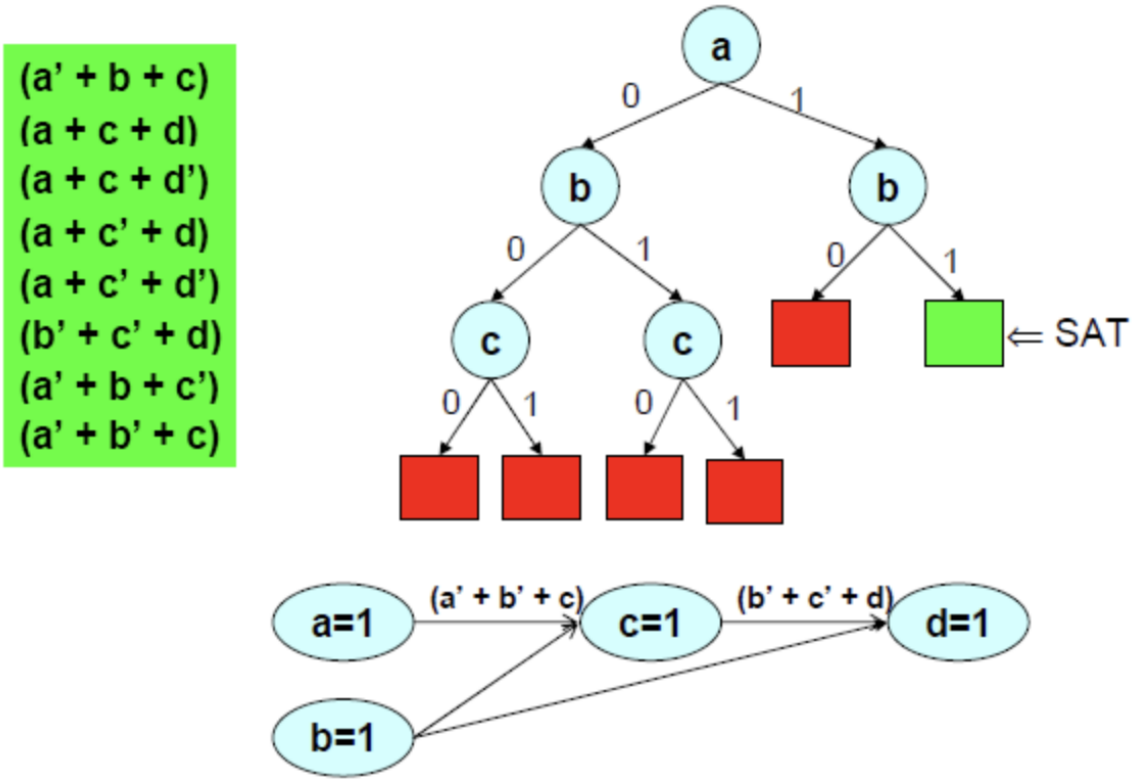


*Figure 1. DPLL search-tree execution source - Wikipedia. Each node represents a partial assignment; leaves marked **SAT** or **UNSAT** are terminal states reached after Unit Propagation.*

2.5 Completeness & Extension to CDCL

DPLL is *complete*: it always terminates and correctly reports SAT or UNSAT. Modern solvers extend DPLL with **Conflict-Driven Clause Learning (CDCL)**: when a contradiction is found, the conflict graph is analysed and a *learned clause* is added to prune future search, yielding dramatic practical speed-ups. Z3 uses a CDCL engine at its core.

3. Exercise 2 — Z3 SMT Solver

3.1 What is Z3?

Z3 is an open-source **Satisfiability Modulo Theories (SMT)** solver developed by Microsoft Research. It extends CDCL-based SAT solving with support for theories such as linear arithmetic, arrays, bit-vectors, and uninterpreted functions. For pure propositional logic, Z3 uses its CDCL engine directly.

3.2 Installation

```
$ pip install z3-solver
```

3.3 The Formula: $p \wedge \neg\neg(\neg q \wedge \neg\neg p)$

We simplify the formula manually before coding it.

$$
\begin{aligned}
& p \wedge \neg\neg(\neg q \wedge \neg\neg p) && \\
\equiv\ & p \wedge \neg\neg(\neg q \wedge p) && \text{(eliminate } \neg\neg p\text{)} \\
\equiv\ & p \wedge (\neg q \wedge p) && \text{(eliminate outer } \neg\neg\text{)} \\
\equiv\ & p \wedge \neg q \wedge p && \text{(expand)} \\
\equiv\ & p \wedge \neg q && \text{(idempotency: } p \wedge p \equiv p\text{)}
\end{aligned}
$$

CNF: $(p) \ \wedge \ (\neg q)$

Unique satisfying model: $p = \mathit{True}, \quad q = \mathit{False}$
Verification: $\top \ \wedge \ \neg\bot = \top \ \wedge \ \top = \top$ ✓

3.4 Python Implementation

```python
from z3 import Bool, And, Not, Solver, sat

# Declare propositional variables
p = Bool('p')
q = Bool('q')

# Build formula: p /\ ~~(~q /\ ~~p)
formula = And(
    p,
    Not(Not(And(
        Not(q),
        Not(Not(p))
    )))
)

# Create solver and check
s = Solver()
s.add(formula)
result = s.check()

if result == sat:
    model = s.model()
    print("Result : SAT")
    print("Model  :", model)
    for decl in model.decls():
        print(f"  {decl.name()} = {model[decl]}")
else:
    print("Result : UNSAT")
```

3.5 Expected Output

```
Result :  SAT
Model :  [q = False, p = True]
         q = False
         p = True
```

3.6 Manual Verification

Given $p = \top,\ q = \bot$:

Step	Sub-expression	Value
1	$\neg q$	$\neg\bot = \top$
2	$\neg\neg p$	$\neg\neg\top = \top$
3	$\neg q \wedge \neg\neg p$	$\top \wedge \top = \top$
4	$\neg\neg(\cdots)$	$\top$
5	$p \wedge \top$	$\top \wedge \top = \top$ ✓

3.7 Algorithm for DPLL-based Z3 Resolution

The following algorithm captures how Z3 internally resolves the given formula using its CDCL (DPLL-based) engine before applying theory reasoning.

Algorithm 2 Z3 CDCL Resolution of $p \wedge \neg\neg(\neg q \wedge \neg\neg p)$

Input: Formula $\varphi = p \wedge \neg\neg(\neg q \wedge \neg\neg p)$
Output: SAT with model, or UNSAT

$\varphi' \leftarrow \text{toCNF}(\varphi)$ ▷ Apply double-neg elim & simplification $\Rightarrow (p) \wedge (\neg q)$
$\alpha \leftarrow \{\}$ ▷ Empty assignment
while $\exists$ unit clause (ℓ) in φ' **do**
  $\alpha \leftarrow \alpha \cup \{\text{var}(\ell) \mapsto \text{pol}(\ell)\}$
  $\varphi' \leftarrow \text{simplify}(\varphi', \ell)$
end while ▷ UP forces $p = \top$ from clause (p), then $q = \bot$ from $(\neg q)$
if $\varphi' = \emptyset$ **then return SAT**, $\alpha = \{p \mapsto \top,\ q \mapsto \bot\}$
end ifreturn UNSAT

3.8 Comparison: Linear Solver vs. DPLL vs. Z3

Approach	Supports $\vee$?	Complexity	Backtracking?
Linear Solver	No (Horn / DAG only)	$O(\lvert V\rvert + \lvert E\rvert)$	No
DPLL	Yes (general CNF)	Exponential (worst)	Yes
Z3 (CDCL+SMT)	Yes + theories	Exp. (CDCL pruning)	Yes + learning

Table 1. Summary of satisfiability approaches discussed in Assignment 1.

4. Terminal-Outputs & Submission

```python
def simplify(formula, literal):
    new_formula = []
    for clause in formula:
        if literal in clause:
            continue
        new_clause = [l for l in clause if l != -literal]
        new_formula.append(new_clause)
    return new_formula


def unit_propagate(formula, assignment):
    changed = True
    while changed:
        changed = False
        for clause in formula:
            if len(clause) == 0:
                return None, None  # Empty clause = contradiction
            if len(clause) == 1:
                unit = clause[0]
                var = abs(unit)
                val = unit > 0
                if var in assignment:
                    if assignment[var] != val:
                        return None, None  # Contradiction
                else:
                    assignment[var] = val
                    formula = simplify(formula, unit)
                    changed = True
                    break
    return formula, assignment

```

Figure 2. *DPLL-Code Snapshot-1 showing simplification*

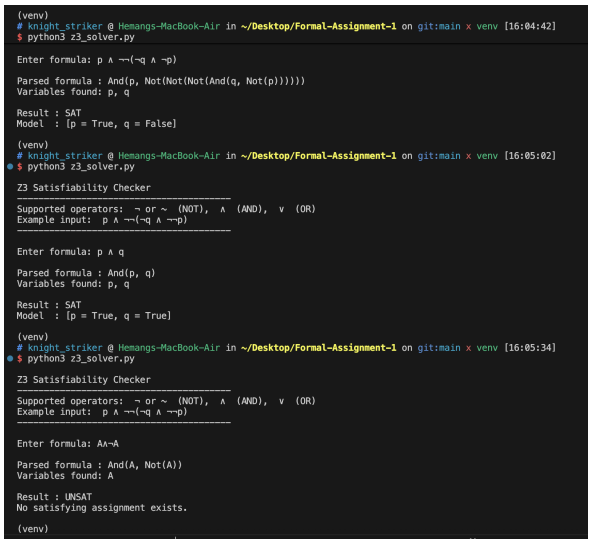
```
# knight_striker @ Hemangs-MacBook-Air in ~/Desktop/Formal-Assignment-1 on git:main x [15:21:01]
$ python3 dpll.py
  DPLL Algorithm

[Example 1]
Formula: (A ∨ B) ∧ (¬A ∨ C) ∧ (¬B ∨ ¬C)
  CNF Formula: (A ∨ B) ∧ (¬A ∨ C) ∧ (¬B ∨ ¬C)
  Result : SAT
  Model  : {'A': True, 'C': True, 'B': False}

[Example 2]
Formula: (P) ∧ (¬P)
  CNF Formula: (P) ∧ (¬P)
  Result : UNSAT

[Example 3 — Assignment formula]
Formula: p ∧ ¬¬(¬q ∧ ¬¬p)
Simplified CNF: (p) ∧ (¬q) ∧ (p)
  CNF Formula: (p) ∧ (¬q) ∧ (p)
  Result : SAT
  Model  : {'p': True, 'q': False}
  DPLL Complete.
```

Figure 3. *Terminal-Output-1 from the DPLL algorithm*

~/Desktop/Formal-Assignment-1/z3_solver.py - Modified

```python
import re
import sys

def parse_formula(expr, variables):
    expr = expr.strip()

    # Remove outer parentheses
    while expr.startswith('(') and expr.endswith(')'):
        depth = 0
        balanced = True
        for i, c in enumerate(expr):
            if c == '(': depth += 1
            elif c == ')': depth -= 1
            if depth == 0 and i < len(expr) - 1:
                balanced = False
                break
        if balanced:
            expr = expr[1:-1].strip()
        else:
            break

    # Handle ¬ (negation) at the start
    if expr.startswith('¬') or expr.startswith('~'):
        return Not(parse_formula(expr[1:].strip(), variables))

    for op_sym, op_fn in [('∨', Or), ('∧', And)]:
        depth = 0
        for i in range(len(expr) - 1, -1, -1):
            if expr[i] == ')': depth += 1
            elif expr[i] == '(': depth -= 1
            elif depth == 0 and expr[i] == op_sym:
                left  = expr[:i].strip()
                right = expr[i+1:].strip()
                return op_fn(parse_formula(left, variables),
                             parse_formula(right, variables))
```

Figure 4. *Code for the z3 solver solver where we can put any outputs and it prints the result whether it is SAT or UNSAT*



```
(venv)
# knight_striker @ Hemangs-MacBook-Air in ~/Desktop/Formal-Assignment-1 on git:main x venv [16:04:42]
$ python3 z3_solver.py

Enter formula: p ∧ ¬¬(¬q ∧ ¬p)

Parsed formula : And(p, Not(Not(Not(And(q, Not(p))))))
Variables found: p, q

Result : SAT
Model  : [p = True, q = False]

(venv)
# knight_striker @ Hemangs-MacBook-Air in ~/Desktop/Formal-Assignment-1 on git:main x venv [16:05:02]
$ python3 z3_solver.py

Z3 Satisfiability Checker
----------------------------------------
Supported operators:  ¬ or ~  (NOT),  ∧  (AND),  ∨  (OR)
Example input:  p ∧ ¬¬(¬q ∧ ¬¬p)
----------------------------------------

Enter formula: p ∧ q

Parsed formula : And(p, q)
Variables found: p, q

Result : SAT
Model  : [p = True, q = True]

(venv)
# knight_striker @ Hemangs-MacBook-Air in ~/Desktop/Formal-Assignment-1 on git:main x venv [16:05:34]
$ python3 z3_solver.py

Z3 Satisfiability Checker
----------------------------------------
Supported operators:  ¬ or ~  (NOT),  ∧  (AND),  ∨  (OR)
Example input:  p ∧ ¬¬(¬q ∧ ¬¬p)
----------------------------------------

Enter formula: A∧¬A

Parsed formula : And(A, Not(A))
Variables found: A

Result : UNSAT
No satisfying assignment exists.

(venv)
```

Figure 5. *Terminal outputs for the z3 solvers*

5. Conclusion

This report covered the theoretical foundations of propositional satisfiability and the DPLL algorithm. The key findings are:

- CNF is the universal input format for DPLL and modern SAT solvers; any propositional formula can be systematically converted to it.
- DPLL uses Unit Propagation, Pure Literal Elimination, and systematic backtracking to decide SAT/UNSAT completely and correctly.
- The formula $p \wedge \neg\neg(\neg q \wedge \neg\neg p)$ simplifies to $p \wedge \neg q$ in CNF, satisfiable with $p = \top,\ q = \bot$.
- Z3 extends DPLL/CDCL to full SMT solving and confirmed the SAT result, returning the correct model $[p = \mathit{True},\ q = \mathit{False}]$.

All the files are submitted as a zip file